CPG Analytics — Specification Package

> **Audience:** engineering + content teams.
> **Status:** DRAFT v0.1 — for team review (rev 2026-06-29).
> These are the contracts to build against. Discussion/rationale lives in `../ux.md`.

How to read this package

File	What it defines	Primary consumer
`00-architecture.md`	Governing principle, 3-tier resolution, system context	everyone
`10-persona-card.schema.md`	**The common Persona Card schema (all levels)**	eng + content
`11-catalog.schema.md`	Shared preference-catalog definitions (query parameters)	eng + content
`12-metric-registry.md`	Metric id → Cube.js measure mapping	eng + data
`13-insight-rule.schema.md`	Insight rule registry (triggers, thresholds, evidence)	eng + data
`14-section-grammar.schema.md`	Home-page layout grammar (render spec)	eng (frontend)
`20-resolution.md`	Runtime resolution: Persona → Tenant → Individual	eng (backend)
`examples/*.yaml`	Real authored instances to load & validate	eng + QA

Read top to bottom on first pass. `10` is the anchor — everything references it.

Locked design decisions (v0.1)

These shape every schema. Reversible, but treat as the baseline.

1. **One schema per concept, common across ALL hierarchy levels.**
   The difference between a Sales Officer and a National Sales Manager is *values in the same shape*, never a different structure. Hierarchy/grain/content-depth are **fields**, not structural branches.
2. **Persona Cards are SEMANTIC; layout is separate.**
   A card declares KPIs, insight types, catalog defaults, questions, actions, hierarchy. It does **not** declare pixel layout. A separate, also-common **section grammar** (`14`) describes the home page and consumes the card.
3. **Cards are THIN; they REFERENCE central registries.**
   `kpis` / `insight_types` reference ids in the metric registry (`12`) and insight-rule registry (`13`). Fix a threshold or measure once → all personas inherit it.
4. **Catalog preference DEFINITIONS are shared; cards carry only overrides.**
   Preference `options` / `maps_to` are industry-generic and live in the catalog (`11`).
   A card carries only `{preference, default, ask_priority}` for the preferences it cares about.

Conventions

- **Format:** authored artifacts are YAML; every schema has a formal **JSON Schema** block for validation. Wire JSON Schema validation into CI — no card/catalog/rule ships unvalidated.
- **IDs:** lowercase snake_case, stable, vendor-owned (`asm` , `secondary_net_sales` , `distributor_underperformance`). IDs are contracts; never reuse or repurpose.
- **Versioning:** each schema file has a `version` . Authored artifacts carry the schema version they target. Vendor card upgrades bump the card's own `version` .
- **The governing principle (non-negotiable):** facts are deterministic (Cube.js + SQL + Python); the LLM only narrates. No schema may let the LLM produce a metric value, a score, or bypass RBAC.

Glossary

- **Persona Card** — vendor-curated, industry-generic blueprint for one role (the IP).
- **Tenant Binding** — a buyer assigning a real user to a persona + injecting their data + RBAC.
- **Individual Model** — per-person learned preferences/behavior layered on top.
- **Catalog preference** — a query parameter the system may need resolved (e.g. `sales_type`).
- **Operating grain** — the dimension a role is accountable for (outlet / distributor / region / nation / brand).
- **Hierarchy tree** — which org tree the role sits in (sales / marketing / analytics).

Build order suggested for the team

1. `12` metric registry + `11` catalog → the lowest-level contracts.
2. `10` persona card schema + `examples/asm.card.yaml` → validate authoring.
3. `13` insight rules → wire deterministic checks against Cube.
4. `20` resolution → assemble Persona → Tenant → Individual at runtime.
5. `14` section grammar → render the resolved payload.